AFRINEXT · PHASE 3 · PAYMENT CHANNEL

Two Words for One Thing, and Why They Must Stay Different

The last field iPayMoney requires is settled: Afrinext launches with mobile money. The value is **quoted from the documentation, not chosen** — and the domain deliberately does not speak it, so no browser can name a provider header.

Branch **claude/create-app-repository-1cxsih** | HEAD **07329e0** | Unit tests **531 passed** | Browser tests **33 passed** | Mutations killed **43 / 45** | Real sandbox calls **0**

THE GATE: THE DOCUMENTATION WAS CONCLUSIVE

The milestone said to stop and ask if the documentation did not establish the exact value, Niger/XOF support, or the field semantics. **It establishes all three**, so no question was sent:

> « *Ipay-Payment-Type* | *string* | *Doit comporter* ***mobile*** *ou* ***card*** »
> — extract **L165** (`POST /api/v1/payments`) and **L238** (`GET /api/v1/payments/{reference}`)

Stated verbatim and **identically on both endpoints**. It is a required HTTP header, type string. The launch value is `mobile` .

A The documentary evidence, line by line

QUESTION THE GATE ASKED	DOCUMENTED	LINE
The exact channel value	« Doit comporter **mobile** ou **card** »	L165, L238
The field semantics	A required HTTP **header**, `Ipay-Payment-Type` , type string, on *both* documented endpoints	L164–168, L234–239
Which operators "mobile money" means	Airtel Money, Zamani Cash, Moov Money — the three Niger operators	L254, L256–257
The currency	`currency` · string · **XOF**	L172
The country code	« **NE** pour le Niger »	L173
Whether the sandbox exercises it	« les numeros de test à utiliser pour les paiements **mobile** »	L147
The cost	3 % on each mobile-money collection	L271

THE ONE THING NOT PRINTED, RECORDED AS ASSUMPTION A13

The documentation gives no **per-country channel matrix**. Mobile money for Niger/XOF is inferred from four converging facts — the operators are Niger's, the currency is XOF, the country example is NE, and the sandbox numbers are for mobile — rather than from a single sentence saying so. That is recorded as **A13** in the assumptions register instead of being asserted, and a real sandbox call is what confirms it.

B Two vocabularies, and why the difference is the security property

LAYER	WORD
Afrinext's domain, and anything a browser posts	`mobile_money`
iPayMoney's header	`mobile`

```
browser ──"mobile_money"──▶ parsePaymentChannel()
                                  refuses everything not in the allowlist,
                                  INCLUDING "mobile" and "card"
                                           │
                                           ▼
                            IPAYMONEY_PAYMENT_TYPE[channel] ──▶ "mobile"
                                  the only place this literal exists
```

A browser **cannot** hand iPayMoney a header value of its choosing — not because the value is filtered, but because **the domain does not speak that vocabulary at all.** Someone who reads `mobile` out of this repository, or out of iPayMoney's public documentation, and posts it is refused exactly as firmly as someone posting nonsense.

That is a stronger guarantee than validation. A filter can be bypassed by finding an unfiltered path; a vocabulary the domain never accepts has no path at all.

> **`CARD` IS ABSENT ON PURPOSE**
>
> It is a *documented* iPayMoney value, and it is deliberately not in the mapping. Afrinext does not offer card payments at launch, and a mapping entry for a channel the business has not adopted is a route to sending one. A test asserts the mapping's values do not contain it.

One undocumented path removed

`createCharge` used to read `?? metadata["paymentType"]` when no channel was given. It was **unreachable** — the order domain builds that metadata and never set the key — but it was a second, undocumented way to set a provider header, which is exactly the hidden default this integration must not have. It is gone, and a mutation restores it to prove something notices.

C Files, and the flow

Two commits on `claude/create-app-repository-1cxsih`, neither merged to `main`: `07329e0` implements the channel (27 files, +601 −86), and `c54b7e3` adds the one assertion the mutation matrix showed was missing (section F).

FILE	Δ	WHY
packages/core/src/payments/channel.ts	+63 new	The allowlist, the parser, the error. The whole domain representation
packages/core/src/payments/channel.test.ts	+87 new	9 tests, including the documented value and the provider-vocabulary refusal
packages/core/src/payments/ipaymoney.ts	+56 −8	`IPAYMONEY_PAYMENT_TYPE` with its L165/L238 citation; the metadata fallback removed
packages/core/src/payments/provider.ts	+10 −1	`ChargeInput.channel` narrowed from `string` to the allowlist type
packages/core/src/orders/payment.ts	+25 −2	`channel` required; validated before the payment row is written
.../orders/checkout-provider-wiring.test.ts	+85 −32	Three tests rewritten for the new contract, two added
apps/web/src/components/PayButton.tsx	+49 −5	The visible, checked radio built from the allowlist
apps/web/src/app/[locale]/orders/[id]/page.tsx	+38 −2	An exhaustive label record, so adding a channel fails the build until it is named
apps/web/src/app/api/v1/orders/[id]/pay/route.ts	+18 −6	Reads and validates the channel from the JSON body
apps/web/src/lib/order-actions.ts	+12 −2	Same, from the form
apps/web/e2e/checkout.spec.ts	+76 −4	Two new browser tests: DOM tampering, and the API's refusals
e2e/{buyer-profile,delivery,refunds}	+19 −2	The shared `chooseMobileMoney` step
core tests (content, orders, refunds, late-payment, ipaymoney)	+32 −21	Call sites given the launch channel — every one found by the compiler, not by searching
docs/architecture/payment-channel.md	+86 new	The decision, the evidence, the rules
docs/providers/ipaymoney/README.md	+37 −4	All four fields now supplied; assumption A13
packages/i18n/src/messages/{fr,en}.json	+10 −2	Four strings each

```
buyer taps pay
  └─ form posts  { orderId, channel: "mobile_money" }   Afrinext's word
       │
       ├─ payOrderAction / POST /api/v1/orders/{id}/pay
       │     parsePaymentChannel(untrusted)   refuses anything else
       │
       └─ initiatePayment({ orderId, channel })
            ├─ parsePaymentChannel again      for untyped callers
            ├─ requireCompleteProfile()
            ├─ writes the payment row
            └─ createCharge({ channel, … })
                 └─ IPAYMONEY_PAYMENT_TYPE[channel]
                      └─ header  Ipay-Payment-Type: mobile
```

The amount is still read from the order row, the phone from the verified sign-in number, and the name and country from the buyer's profile. **The channel is the only new thing a person can influence**, and it is checked twice before it is used.

D Every way a channel can be wrong

WHAT ARRIVES	RESULT	PROVIDER REQUESTS	PAYMENT ROWS
`"mobile_money"`	ACCEPTED	1	1
`"mobile"`	REFUSED the provider's own word	0	0
`"card"`	REFUSED documented, not offered	0	0
`"ussd"` · `"bank_transfer"`	REFUSED	0	0
`"MOBILE_MONEY"` · `" mobile_money"`	REFUSED exact match only	0	0
absent · `null` · `""`	REFUSED never defaulted	0	0
`1` · `{}` · `["mobile_money"]` · `true`	REFUSED	0	0

Every refusal happens **before the payment row is written**, so an invalid channel leaves nothing behind — no row for `payments_one_live_per_order` to wedge, and nothing to clean up. The buyer picks again and pays the same order.

Two layers do this and both are load-bearing:

- **The type.** `InitiatePaymentInput.channel` is required and typed as the allowlist, so a caller that omits or mistypes it **fails to compile**. That is how every existing call site in the codebase was found — the compiler listed them.
- **The runtime check.** `parsePaymentChannel` catches what the type cannot: a JSON body, a form field, a caller compiled against an older shape.

E One option, still a visible choice

Afrinext offers exactly one channel, and it is rendered as a **checked radio naming its operators** — not a hidden input.

```
Moyen de paiement
  (•) Mobile Money
      Airtel Money, Zamani Cash ou Moov Money
  C'est le seul moyen de paiement disponible au lancement.
```

A hidden field would have been less code and worse: **a payment method the buyer never saw is not a method they chose**, and it would make the single option look like an implementation detail rather than a decision. The radio also means a second channel needs no new control.

The list is built from `PAYMENT_CHANNELS`, so the screen can only offer what the server accepts — there is no second list to forget. The label lookup is a `Record` over the allowlist rather than a template-literal key, so **adding a channel fails the build** until somebody names it in both catalogues. A dynamic key would have compiled and shipped a screen showing a raw identifier.

F Mutation results

Ten deliberate defects, each one a rule this milestone names, run against the full `@afrinext/core` suite.

#	DEFECT INTRODUCED	FIRST RUN	AFTER
C1	The allowlist also accepts `mobile` and `card`	KILLED	KILLED
C2	The parser accepts any non-empty string	KILLED	KILLED
C3	A missing channel silently defaults to the launch channel	KILLED	KILLED
C4	Matching becomes case-insensitive and trims	KILLED	KILLED
C5	`card` added to the provider mapping	KILLED	KILLED
C6	The header value becomes `mobile_money` instead of the documented `mobile`	KILLED	KILLED
C7	The mapping is no longer frozen	KILLED	KILLED
C8	**The `metadata["paymentType"]` fallback is restored**	SURVIVED	KILLED
C9	The runtime validation in `initiatePayment` is removed	KILLED	KILLED
C10	The channel is validated *after* the payment row is written	KILLED · 162 failing	KILLED

C8 SURVIVED, AND IT IS THE SAME LESSON AS LAST MILESTONE

Restoring the metadata fallback walked through a fully green suite — for exactly the reason three mutants did in the buyer-profile milestone. The order domain never sets `metadata.paymentType`, and `initiatePayment` always supplies a validated channel, so **the fallback is unreachable from a checkout.** Every test starts from a checkout.

Unreachable from there is not unreachable. `ChargeInput.channel` is optional at the *adapter* boundary, so a direct caller can omit it — and with the fallback in place, metadata would then decide a provider header. That is precisely the hidden default this milestone exists to prevent.

It is now asserted where it is reachable: `createCharge` with no channel and a `metadata.paymentType` refuses and constructs nothing. **No production behaviour changed** — only whether something checks it.

C10 is worth a glance for the opposite reason: moving the validation after the row write failed **162** tests. A guarantee that many things depend on is one it is hard to remove quietly.

The other matrices, unchanged

MATRIX	RESULT	AGAINST BASELINE
Channel (new)	`10 / 10 KILLED`	First run 9 of 10 — see C8
Buyer profile	`9 / 9 KILLED`	unchanged
Refund execution	`24 / 26 KILLED`	unchanged · R7/R8 redundant by design, R26 kills all three

G Results, against the previous baseline

CHECK	RESULT	BASELINE	DELTA EXPLAINED
`pnpm lint`	CLEAN	clean	—
`pnpm typecheck`	CLEAN	clean	—
`pnpm test`	531 PASSED · 16 SKIPPED · 28 files	520 · 16 · 27	**+11**: 9 channel tests, 2 wiring tests. No existing test removed or weakened
`pnpm build`	COMPILED	compiled	—
`playwright`	33 PASSED	31 passed	**+2**: DOM tampering, and the API refusing five bad channels
`reconcile-check`	CLEAN	clean	drift 0 · 0 unbalanced · net XOF 0 · both probes firing
CI	SUCCESS	success	Run #56 on `07329e0`, all three jobs green; run #58 on `c54b7e3`

What the compiler found for us

Making `channel` required turned every existing call site into a compile error — in `content`, `orders`, `late-payment`, `refunds` and the adapter's own fixtures. That list came from the compiler rather than from grepping, which is the difference between "I updated the call sites" and "the call sites are updated".

Three tests in the wiring suite were rewritten because their subject no longer exists: they asserted behaviour for an *omitted* channel, which is now a type error rather than a runtime path. Their replacements assert the reachable cases — a missing channel from an untyped caller, six unsupported values including the provider's own two, and six non-string values.

Untouched, and verified so

SUITE	TESTS	STATE
OTP and phone auth	26	UNMODIFIED
Refund execution	80	CALL SITES GIVEN THE CHANNEL; ASSERTIONS UNMODIFIED
Late payment	31	SAME
iPayMoney adapter · webhook	44	FIXTURE CHANNEL UPDATED; ONE ASSERTION ADDED
Replay	5	UNMODIFIED
Buyer profile	21	UNMODIFIED

Settlement, the ledger, payout logic, the refund state machine, the late-payment state machine, webhook verification, OTP and signup are all **byte-identical**.

H Blockers, and what comes next

#	BLOCKER	WHOSE	STATE
S9	No payment channel chosen	Product	CLOSED by this milestone
S8	No buyer name collected	Afrinext	CLOSED
S7	Checkout does not pass the buyer's details	Afrinext	CLOSED
S6	**No sandbox credentials.** No real payment has ever been made	External	OPEN — NOW THE ONLY THING IN THE WAY
S1	Phase 3 has not been approved	You	OPEN
S2	Settlement timing T+N — K16	iPayMoney	OPEN
S3	The amount is not verifiable from the provider — K8	iPayMoney	OPEN
S4	The 3 % fee's treatment on refund — K11	iPayMoney	OPEN
S5	No customer-refund execution — K1, K5	iPayMoney	OPEN

EVERY FIELD IPAYMONEY REQUIRES IS NOW SUPPLIED

FIELD	SOURCE
`msisdn`	The number that answered an OTP at sign-in
`customer_name`	`users.full_name` — what the buyer typed
`country`	`users.country_code` — what the buyer chose
`Ipay-Payment-Type`	`mobile`, mapped from the chosen `mobile_money`
`amount · currency · transaction_id`	Server-derived from the order, unchanged

The adapter can now construct a complete, well-formed charge. What it has never done is send one.

Yes — sandbox credentials are the right next step

Nothing in the code stands between here and a real call. Sixteen tests are written, gated on `IPAYMONEY_BASE_URL`, `IPAYMONEY_API_KEY` and an explicit `IPAYMONEY_ENVIRONMENT=sandbox`, and they answer questions no amount of local testing can:

- **A13** — that mobile money is in fact available for Niger and XOF, which the documentation implies across four separate facts but never states in one sentence.

- **K12** — the real `status` string a declined or insufficient-funds payment carries. The adapter refuses an undocumented status rather than guessing, so the first run reports the exact value.
- **K8** — whether an amount comes back after all, which would change `statesChargeAmount` in our favour.
- **K13** — that `"5000"` means 5 000 francs and not 50. A factor of 100 on real money.

The gate cannot fall back to Live: `IPAYMONEY_ENVIRONMENT` has no default at all, and a missing or mistyped value skips rather than proceeds. **K1–K18 remain outstanding**; none was answered, inferred or modified by this milestone.